SENTINELSHIELD:

ADVANCED INTRUSION DETECTION & WEB PROTECTION SYSTEM

Practical Work Documentation

Student Name:

Vaishali Vasant Kadam

Project Title:

SentinelShield :

Advanced Intrusion Detection & Web Protection System

Submitted For:

Internship Practical Documentation

Submission Date:

22 March 2026

GitHub :

https://github.com/kadamvaishali378/SentinelShield

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the organization and mentors who provided me the opportunity to complete my cybersecurity internship and work on the project titled **“SentinelShield: Advanced Intrusion Detection & Web Protection System.”**

This project provided valuable practical exposure to the field of web application security and intrusion detection systems. Through this project, I was able to understand how modern security monitoring systems inspect incoming web traffic and identify malicious activities.

I would also like to thank my internship mentors and the organization for their guidance, support, and encouragement throughout the development of this project. Their valuable insights helped me successfully complete this practical work.

Finally, I would like to thank everyone who supported me during the completion of this project.

DECLARATION

I hereby declare that the project titled **“SentinelShield: Advanced Intrusion Detection & Web Protection System”** submitted as part of my internship practical work is my original work.

All the information used in this document has been properly acknowledged wherever necessary. This report has been prepared for educational purposes to demonstrate the implementation of web security monitoring techniques.

Student Name: Vaishali Vasant Kadam
Date: 22 March 2026

TABLE OF CONTENTS

TABLE OF FIGURES

1. INTRODUCTION

Web applications play an important role in modern digital systems. Organizations depend on web applications for services such as online banking, e-commerce, communication platforms, and data management. Because of their widespread usage, web applications are frequently targeted by cyber attackers.

Attackers attempt to exploit vulnerabilities in web applications using different techniques such as:

- SQL Injection
- Cross-Site Scripting (XSS)
- Directory Traversal
- Command Injection

If these attacks are successful, attackers may gain unauthorized access to databases, steal sensitive information, or even compromise the entire system.

To protect web applications, organizations deploy security solutions such as Web Application Firewalls (WAF), Intrusion Detection Systems (IDS), and security monitoring tools. These systems analyze incoming HTTP requests and detect suspicious patterns that may indicate malicious activity.

The **SentinelShield system** developed in this project simulates the basic functionality of such security monitoring systems. It inspects incoming web requests, detects malicious input patterns, logs security events, and visualizes attack statistics through a monitoring dashboard.

This practical project helps demonstrate the working principles of modern cybersecurity monitoring systems used in real-world environments such as Security Operations Centers (SOC).

2. PROJECT OVERVIEW / DESCRIPTION

SentinelShield is a simplified web security monitoring system designed to analyze incoming HTTP requests and detect malicious activities targeting web applications. The system functions as a lightweight intrusion detection mechanism that inspects request parameters and compares them with predefined attack signatures.

When a request is received, the system extracts the query parameters and analyzes them for suspicious patterns commonly associated with cyber attacks. These patterns include attacks such as Cross-Site Scripting (XSS), SQL Injection, Directory Traversal, and Command Injection. If a malicious pattern is detected, the system classifies the request as malicious and records the event in a security log file for monitoring and investigation.

The system also implements a basic rate limiting mechanism that tracks the number of requests coming from a particular IP address. If too many requests are received within a short period of time, the system identifies the behavior as suspicious. This helps in detecting automated attacks, scanning attempts, or brute-force activities.

Another important component of SentinelShield is the logging system. Every request processed by the application is recorded with details such as timestamp, IP address, request query, request status, and detected attack category. These logs help administrators review system activity and analyze potential threats.

In addition, the system includes a security dashboard that visually summarizes system activity. The dashboard displays statistics such as total requests, malicious traffic, and attack distribution using graphical charts.

Overall, SentinelShield demonstrates the fundamental working principles of web security monitoring systems and provides practical insight into how intrusion detection mechanisms help protect web applications from cyber attacks.

3. PROBLEM STATEMENT

Web applications are widely used for services such as online banking, e-commerce platforms, communication systems, and data management. Because of their widespread use and accessibility through the internet, web applications have become one of the most common targets for cyber attacks.

Attackers often attempt to exploit vulnerabilities in web applications by sending specially crafted HTTP requests containing malicious input. These attacks may include techniques such as SQL Injection, Cross-Site Scripting (XSS), Directory Traversal, and Command Injection. If these attacks are successful, they can lead to unauthorized access to sensitive data, system compromise, or disruption of services.

Many small or experimental web applications do not implement proper security monitoring mechanisms. Without continuous monitoring and inspection of incoming requests, malicious activities may remain undetected, allowing attackers to exploit vulnerabilities and compromise the system.

This project addresses this problem by developing a simplified intrusion detection system called **SentinelShield**. The system inspects incoming HTTP requests, analyzes request parameters, and detects common attack patterns using signature-based detection techniques. By identifying suspicious inputs and logging security events, the system helps demonstrate how security monitoring tools can detect and track malicious web activities.

4. PROJECT OBJECTIVES

The SentinelShield project is designed to demonstrate the basic principles of web security monitoring and intrusion detection in web applications. The system focuses on analyzing incoming HTTP requests and identifying potentially malicious activities that may indicate cyber attacks.

The main objectives of the SentinelShield project are as follows:

• **To inspect incoming HTTP requests sent to the web application**
The system analyzes each HTTP request received by the server. It examines the request parameters to understand the data being sent by the user and checks whether the request contains any suspicious or harmful input.

• **To detect malicious attack patterns in request parameters**
SentinelShield identifies common web attack patterns by comparing request data with predefined attack signatures. This allows the system to detect threats such as Cross-Site Scripting (XSS), SQL Injection, Directory Traversal, and Command Injection attempts.

• **To monitor request behavior and identify suspicious activity**
The system also observes the behavior of incoming requests. By tracking the number of requests coming from a particular IP address, the system can detect unusual patterns such as repeated or rapid requests that may indicate automated attacks or scanning activity.

• **To record security events in log files**
All requests processed by the system are recorded in a log file. These logs include important details such as timestamp, IP address, request parameters, request status, and the detected attack category. This helps in maintaining a record of system activity for monitoring and analysis.

• **To visualize system activity using a security dashboard**
SentinelShield provides a dashboard that displays summarized information about system activity. The dashboard presents data such as total requests, detected attacks, and traffic distribution using graphical charts, making it easier to understand the security status of the system.

Through these objectives, the SentinelShield project demonstrates the fundamental working principles of web security monitoring systems and helps in understanding how intrusion detection mechanisms are used to identify and analyze cyber threats in web applications.

5. SCOPE OF THE PROJECT

The scope of the SentinelShield project is focused on demonstrating the fundamental concepts of intrusion detection and web application security monitoring. The system is designed to simulate how security tools inspect incoming web requests and identify potentially malicious activities targeting a web application.

SentinelShield primarily concentrates on detecting some of the most common web-based cyber attacks that are frequently used by attackers to exploit application vulnerabilities. These attacks include:

• **Cross-Site Scripting (XSS)** – where attackers attempt to inject malicious scripts into web application inputs.
• **SQL Injection** – where attackers try to manipulate database queries by inserting malicious SQL commands.
• **Directory Traversal** – where attackers attempt to access restricted files or directories on the server.
• **Command Injection** – where attackers try to execute system-level commands through application inputs.

The system uses **signature-based detection techniques** to identify these attack patterns. Incoming HTTP request parameters are analyzed and compared with predefined malicious signatures. If a match is found, the system classifies the request as malicious and records the event in the security log.

In addition to signature detection, SentinelShield also implements **behavior-based monitoring** using a rate limiting mechanism. This feature tracks repeated requests from the same IP address within a short period of time. If an unusually high number of requests is detected, the system flags the behavior as suspicious.

6. TOOLS & TECHNOLOGIES USED

The development of the SentinelShield system involved the use of several tools and technologies that helped in building, testing, and monitoring the web security application. These technologies were selected because they are widely used in web development and cybersecurity projects.

Python

Python was used as the primary programming language for implementing the SentinelShield system. It was chosen because of its simplicity, readability, and powerful libraries that support web development and security-related tasks. Python was used to write the main application logic, process HTTP requests, detect malicious patterns, and handle logging of security events.

Flask Framework

Flask is a lightweight web framework used to build web applications in Python. In this project, Flask was used to create the web server that receives and processes HTTP requests from users. It allows the system to define routes such as /inspect and /summary to analyze request parameters and display security statistics. Flask provides a simple structure for handling requests and generating responses.

Python Logging Module

The Python logging module was used to record system activity and security events. Every request processed by the system generates a log entry containing details such as timestamp, IP address, request parameters, and detection results. These logs help in tracking system activity and analyzing potential security threats.

Chart.js

Chart.js is a JavaScript library used for creating graphical charts and visualizations. In the SentinelShield system, Chart.js was used to generate charts for the security dashboard. The dashboard displays visual summaries such as the number of normal requests, detected attacks, and attack distribution. These visualizations make it easier to understand system activity.

Web Browser

A web browser such as **Google Chrome** was used to interact with the application and test the system. HTTP requests were sent through the browser to simulate both normal and malicious inputs. This helped verify whether the system was correctly detecting and classifying different types of attacks.

These tools and technologies together helped in developing a functional web security monitoring system that demonstrates the core concepts of intrusion detection and web application protection.

7. SYSTEM ARCHITECTURE

The SentinelShield system is designed using a **modular architecture**, where multiple components work together to inspect incoming web requests, detect malicious activities, and monitor system behavior. Each module performs a specific function in the request processing pipeline, allowing the system to analyze traffic efficiently and record security events.

The main components of the SentinelShield system include:

- **User Request Interface**
- **Flask Web Server**
- **Request Inspection Engine**
- **Attack Signature Detection Module**
- **Rate Limiting Module**
- **Logging System**
- **Security Dashboard**

The **User Request Interface** represents the client side of the system, typically a web browser such as Google Chrome. The user sends HTTP requests to the web application through the browser. These requests may contain normal input or potentially malicious data intended to exploit application vulnerabilities.

Once a request is sent, it is received by the **Flask Web Server**, which acts as the main server responsible for handling incoming traffic. Flask processes the request and forwards the request parameters to the internal detection modules for further analysis.

The **Request Inspection Engine** examines the request parameters extracted from the URL query or form data. This module prepares the data for analysis by identifying the relevant input fields that may contain malicious payloads.

After inspection, the request data is passed to the **Attack Signature Detection Module**. This module compares the request input against predefined attack patterns such as Cross-Site Scripting (XSS), SQL Injection, Directory Traversal, and Command Injection. If a matching pattern is found, the system classifies the request as malicious and identifies the attack category.

The system also includes a **Rate Limiting Module** that monitors the number of requests received from a particular IP address within a short period of time. If excessive requests are detected, the system marks the activity as suspicious, which may indicate automated scanning or brute-force attempts.

All request activities and detection results are recorded by the **Logging System**. This module stores detailed log entries containing information such as the timestamp, IP address, request parameters, request status, and detected attack type. These logs help in tracking system activity and analyzing potential security incidents.

Finally, the **Security Dashboard** provides a visual representation of system activity. It displays summarized statistics such as the number of total requests, normal traffic, malicious requests, and attack distribution. These visual insights help administrators quickly understand the security status of the system.

Together, these components form the SentinelShield architecture, enabling the system to monitor web traffic, detect threats, and maintain a record of security events for analysis and investigation.

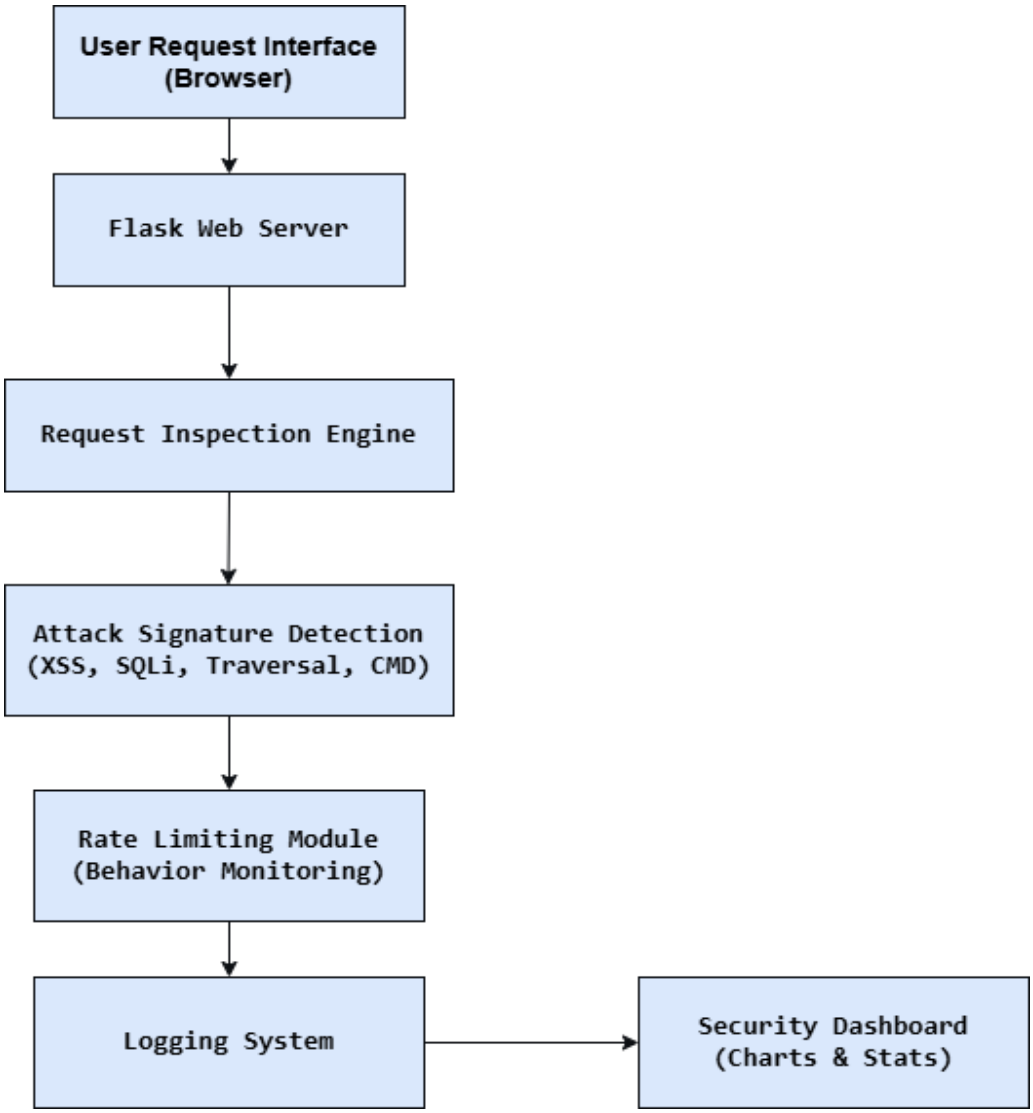


Figure 1: SentinelShield System Architecture

8. WORKFLOW / FLOWCHART

The SentinelShield system processes every incoming HTTP request through a structured workflow to ensure that the request is properly analyzed and classified. This workflow consists of multiple stages where the request is inspected, evaluated for potential threats, and recorded for monitoring purposes.

1. User sends HTTP request
The workflow begins when a user sends an HTTP request to the web application through a web browser. The request may contain different parameters or input values provided by the user. These inputs may be normal data or potentially malicious payloads intended to exploit vulnerabilities in the system.

2. Flask server receives the request
The request is received by the Flask web server, which acts as the main component responsible for handling incoming traffic. Flask processes the request and prepares it for further analysis by the system.

3. Query parameters are extracted
After receiving the request, the system extracts the query parameters or input data from the request URL. These parameters are the main elements that are inspected because attackers often inject malicious code through input fields.

4. Attack signatures are checked
The extracted parameters are analyzed by the attack detection module. The system compares the request data with predefined attack signatures such as patterns associated with Cross-Site Scripting (XSS), SQL Injection, Directory Traversal, or Command Injection. If any matching pattern is found, the request is flagged as suspicious.

5. Rate limiting is applied
At this stage, the system checks how many requests have been received from the same IP address within a specific time period. If too many requests are detected from a single source, the system identifies this as suspicious behavior and may classify the activity as a potential automated attack.

6. Request is classified as normal or malicious
Based on the results of the signature detection and rate limiting checks, the system classifies the request as either a normal request or a malicious request. If malicious patterns are detected, the system also identifies the type of attack.

7. Event is logged in the log file
Once the request is classified, the system records the event in a log file using the logging module. The log entry includes details such as the timestamp, IP address, request query, request status, and attack category.

8. Dashboard statistics are updated
Finally, the system updates the security dashboard with the latest statistics. The dashboard reflects information such as the total number of requests, number of detected attacks, and distribution of attack types, allowing administrators to monitor system activity visually.

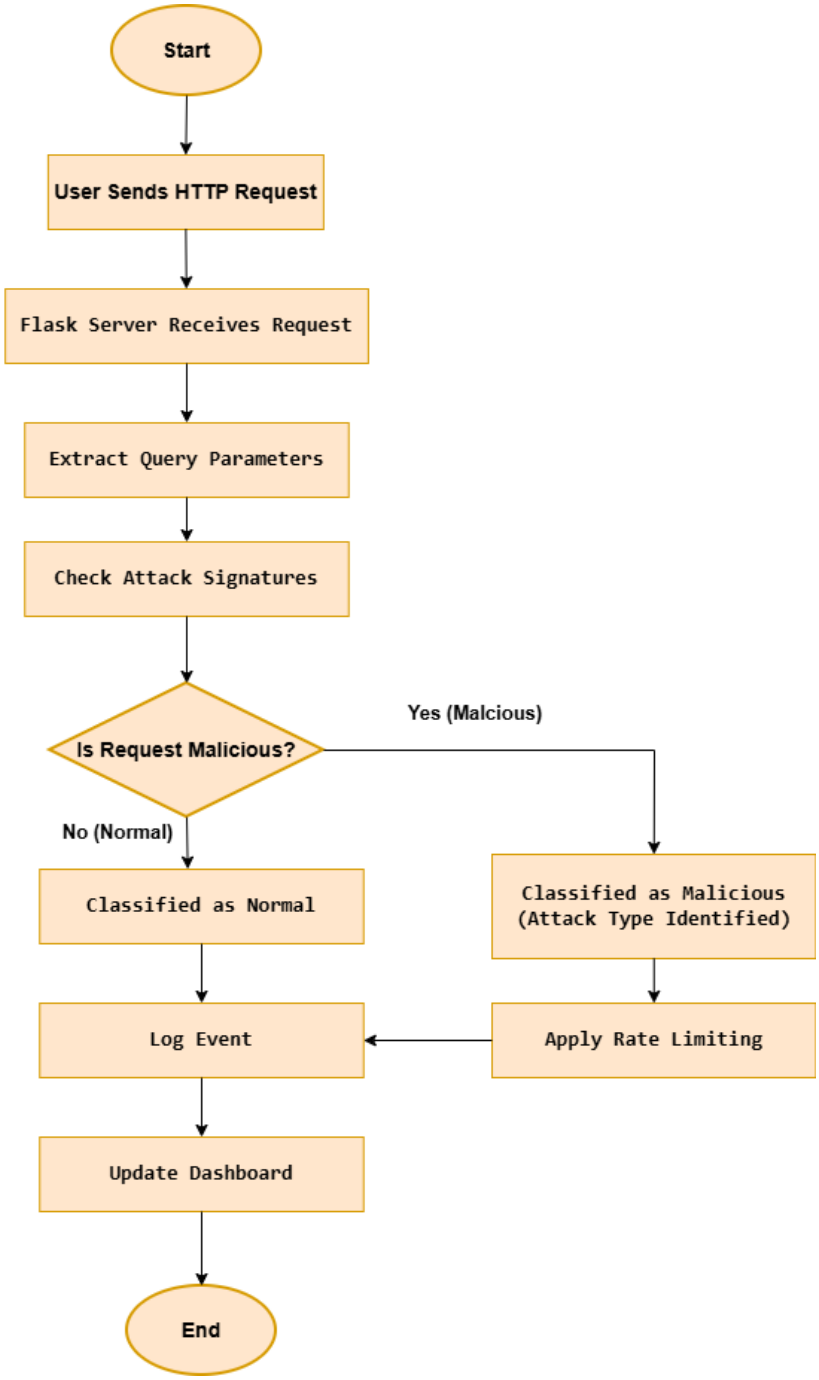


Figure 2: Request Processing Workflow

9. IMPLEMENTATION / CODE DESCRIPTION

This section describes how the SentinelShield system was implemented using Python and the Flask web framework. The implementation focuses on analyzing incoming HTTP requests, detecting malicious patterns, recording security events, and displaying summarized statistics through a monitoring dashboard.

The system consists of several components including the Flask web server, request inspection logic, attack detection rules, rate limiting mechanism, logging system, and dashboard visualization.

Flask Application Initialization

The SentinelShield system begins by initializing a Flask web application.

```
app = Flask(__name__)
```

This statement creates the Flask application instance which acts as the web server. The server listens for incoming HTTP requests from users and processes them using defined routes. Flask is responsible for handling request routing, executing detection logic, and returning responses to the client.

Logging Configuration

The Python logging module is used to record system activity and detected security events.

```
logging.basicConfig(
filename="sentinelshield.log",
level=logging.INFO,
format="%(asctime)s | %(message)s"
)
```

This configuration creates a log file named **sentinelshield.log** where all system events are recorded. Each log entry includes important information such as the timestamp, IP address, request parameters, request status, and attack category. These logs help security analysts track system activity and investigate suspicious behavior.

Request Handling and Route Implementation

The system defines a Flask route that captures incoming HTTP requests for inspection.

Example route:

```
@app.route("/inspect")
def inspect():
```

This route receives user requests containing query parameters. When a request is sent to this endpoint, the system extracts the request data and forwards it to the detection modules for further analysis.

By using Flask routing, the system can process user input dynamically and apply security checks before generating a response.

Request Parameter Extraction

After receiving a request, the system extracts the query parameters from the URL.

Example:

```
query = request.query_string.decode()
ip = request.remote_addr
```

The **query string** represents the user input included in the request. Attackers often inject malicious payloads within these parameters, which is why the system analyzes them carefully.

The system also records the **IP address of the client** sending the request. This information is used by the rate limiting module to monitor repeated requests from the same source.

Attack Signature Detection

The system detects malicious inputs using predefined attack signatures.

Example detection logic:

```
if "<script>" in query:
 category = "XSS"
elif "or 1=1" in query:
 category = "SQL Injection"
elif "../" in query:
 category = "Directory Traversal"
elif "cmd" in query:
 category = "Command Injection"
```

These rules compare the request input with known malicious patterns associated with common web attacks. If a pattern is detected, the system categorizes the request according to the corresponding attack type.

Signature-based detection is widely used in intrusion detection systems and web application firewalls to identify known threats.

Rate Limiting Implementation

In addition to signature detection, the system monitors request behavior using a rate limiting mechanism.

Example logic:

```
ip_tracker[ip] = [timestamps]
```

The system records the timestamps of requests received from each IP address. If an IP sends too many requests within a short period of time, the system flags the activity as suspicious.

This helps detect automated attacks such as:

• Brute-force attempts
• Automated vulnerability scanning
• Bot traffic

Rate limiting therefore acts as a **behavior-based detection technique**.

Security Event Logging

After analyzing the request, the system records the event in the log file.

Example log entry format:

2026-03-14 | IP:127.0.0.1 | Query:/inspect?search=<script> | Status:Malicious | Category:XSS

Each log entry provides detailed information about the request and the detection result. These logs help administrators review attack attempts and analyze patterns of malicious activity.

Dashboard Route for Monitoring

The system also includes a route that displays summarized security statistics through a dashboard.

Example route:

@app.route("/summary")
def summary():

This route processes collected request data and generates statistics such as:

• Total number of requests
• Number of malicious requests
• Distribution of attack types
• Normal vs malicious traffic

These statistics are then visualized using charts on the security dashboard.

Data Processing for Visualization

Before displaying the dashboard, the system processes the collected log data to calculate statistics about system activity.

The processed data includes:

• Total requests received
• Number of detected attacks

• Attack category distribution
• Request frequency

This data is passed to the dashboard interface, where it is displayed using graphical charts created with Chart.js. These visualizations help administrators quickly understand system behavior and monitor potential threats.

10. DETECTION TECHNIQUES IMPLEMENTED

The SentinelShield system uses two main detection techniques to identify malicious activities in incoming HTTP requests. These techniques help the system analyze request content as well as user behavior to detect potential security threats.

Signature-Based Detection

Signature-based detection works by comparing incoming request parameters with a set of predefined malicious patterns or signatures. If a request contains any pattern that matches a known attack signature, the system classifies the request as malicious.

This technique is commonly used in many cybersecurity tools such as antivirus software, intrusion detection systems, and web application firewalls. It is effective for detecting well-known attack types.

Examples of patterns used in the SentinelShield system include:

Pattern	Attack Type
<script>	Cross-Site Scripting (XSS)
or 1=1	SQL Injection
../	Directory Traversal
cmd	Command Injection

When a request parameter contains one of these patterns, the system immediately identifies the corresponding attack category and records the event in the security log.

Behavior-Based Detection

Behavior-based detection focuses on monitoring user activity patterns rather than checking only specific attack signatures. This technique analyzes how frequently requests are sent and identifies unusual behavior that may indicate malicious activity.

Examples of suspicious behavior include:

- Multiple requests sent from the same IP address within a short time period
- Rapid repeated requests to the server
- Automated scanning or bot activity attempting to probe the system

In the SentinelShield system, behavior-based detection is implemented using a **rate limiting mechanism**. The system tracks the number of requests coming from each IP address and monitors the timestamps of those requests. If the number of requests exceeds a predefined threshold within a short time window, the system flags the activity as suspicious.

By combining signature-based detection with behavior-based monitoring, SentinelShield is able to detect both known attack patterns and abnormal request behavior.

11. TESTING METHODOLOGY

The SentinelShield system was tested using different types of HTTP requests to evaluate its ability to detect malicious activities and correctly classify incoming traffic. The testing process involved sending both normal and intentionally malicious inputs to the application to observe how the system responded to each request.

The purpose of the testing phase was to verify whether the attack detection logic, rate limiting mechanism, and logging system were functioning correctly. By performing these tests, it was possible to confirm that the system could successfully identify common web attacks and record them in the log files.

Several test cases were designed to simulate different scenarios:

Normal Request Testing
A standard HTTP request containing normal input was sent to the system. This test ensured that legitimate user requests were processed correctly and not incorrectly flagged as malicious.

Cross-Site Scripting (XSS) Attack Simulation
An input containing a script tag was used to simulate a Cross-Site Scripting attack. The purpose of this test was to verify whether the system could detect script-based injection attempts within the request parameters.

SQL Injection Testing
A request containing a typical SQL injection pattern such as “or 1=1” was sent to the system. This test helped determine whether the attack detection module could identify attempts to manipulate database queries.

Directory Traversal Testing
A request containing patterns such as “../” was used to simulate directory traversal attacks. This test checked whether the system could detect attempts to access restricted files or directories on the server.

Command Injection Testing
Inputs containing command-related patterns such as “cmd” were used to simulate command injection attacks. This test verified whether the system could identify attempts to execute system commands through application inputs.

Each request was processed by the SentinelShield system and analyzed according to the implemented detection rules. The system classified the request as either normal or malicious, and the result was recorded in the log file. By observing the classification results and log entries, the accuracy of the system’s detection capabilities was evaluated.

12. SAMPLE OUTPUTS / RESULTS

Several test requests were used to verify system functionality.

1. Flask Server Running

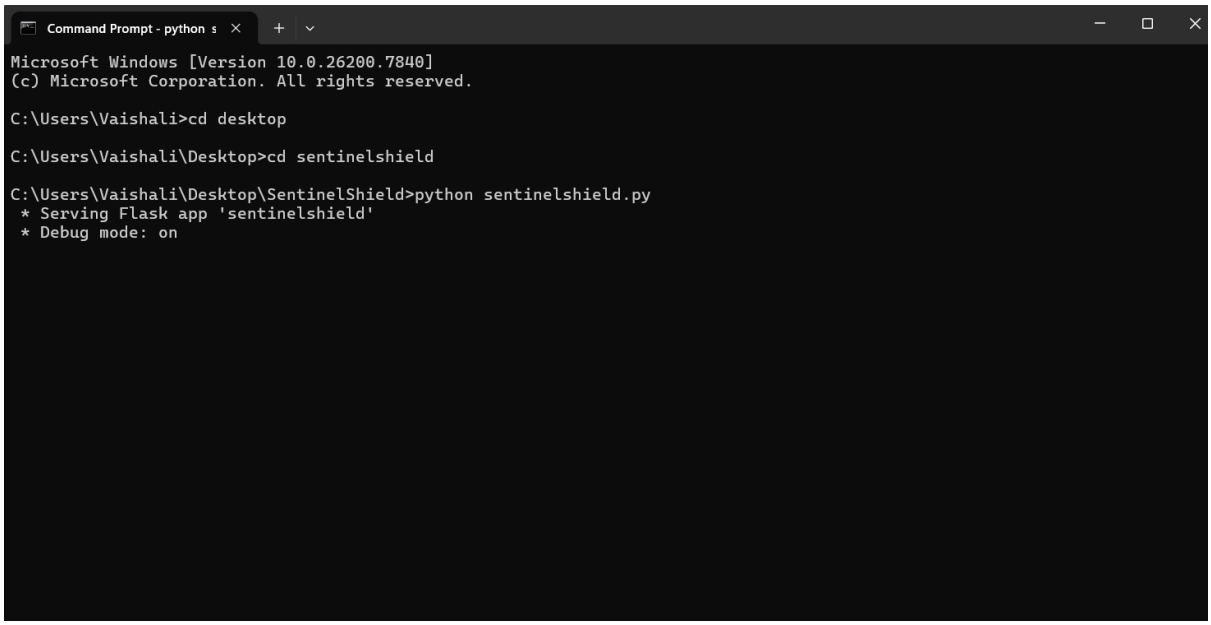


Figure 3: Flask Server Initialization

2. Normal Request

Example:

http://127.0.0.1:5000/inspect?name=hello

Result: Classified as Normal Request

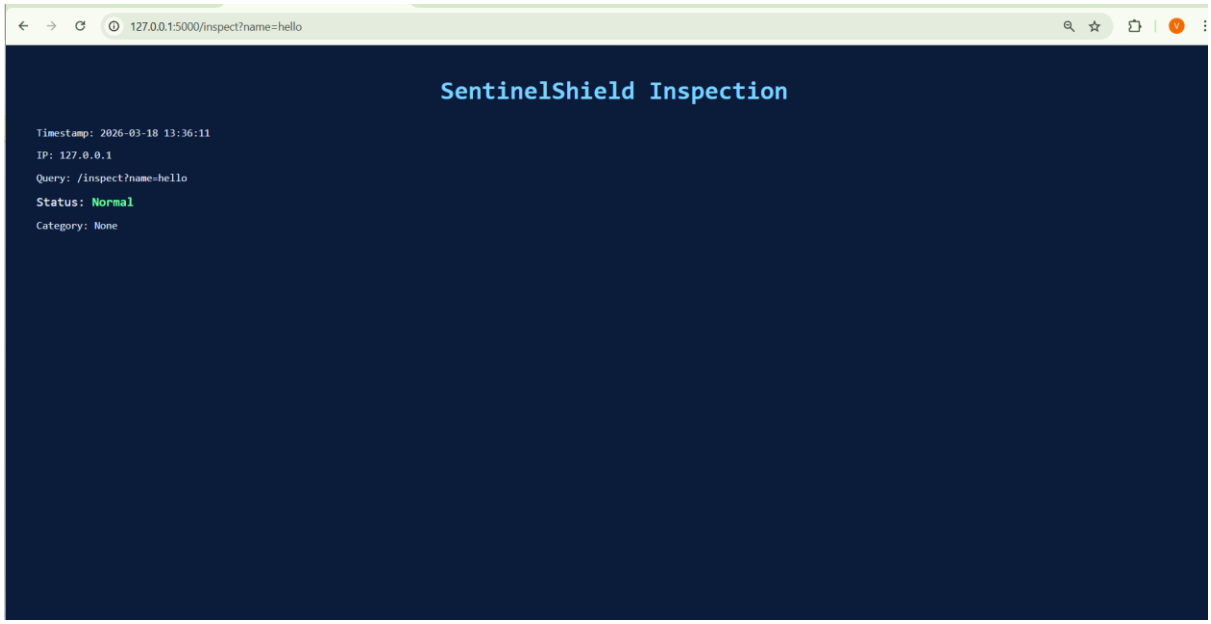


Figure 4: Normal Request Detection

3. XSS Attack

http://127.0.0.1:5000/inspect?search=<script>alert(1)</script>

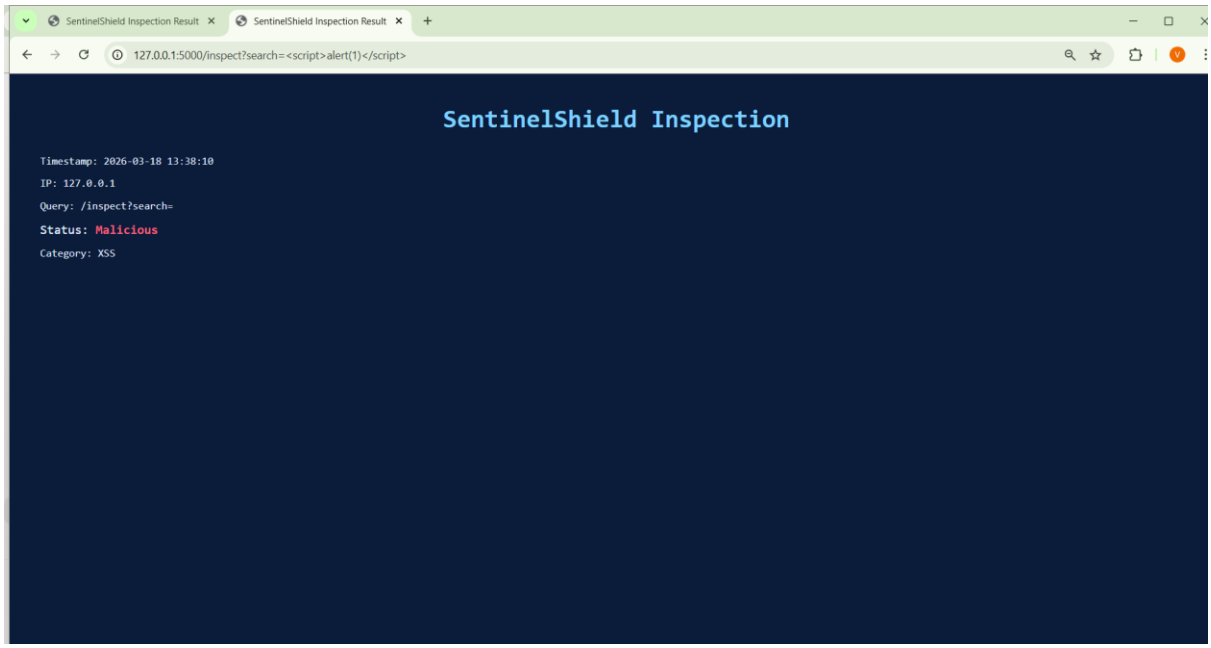


Figure 5: XSS Detection

4. SQL Injection

http://127.0.0.1:5000/inspect?id=1 or 1=1

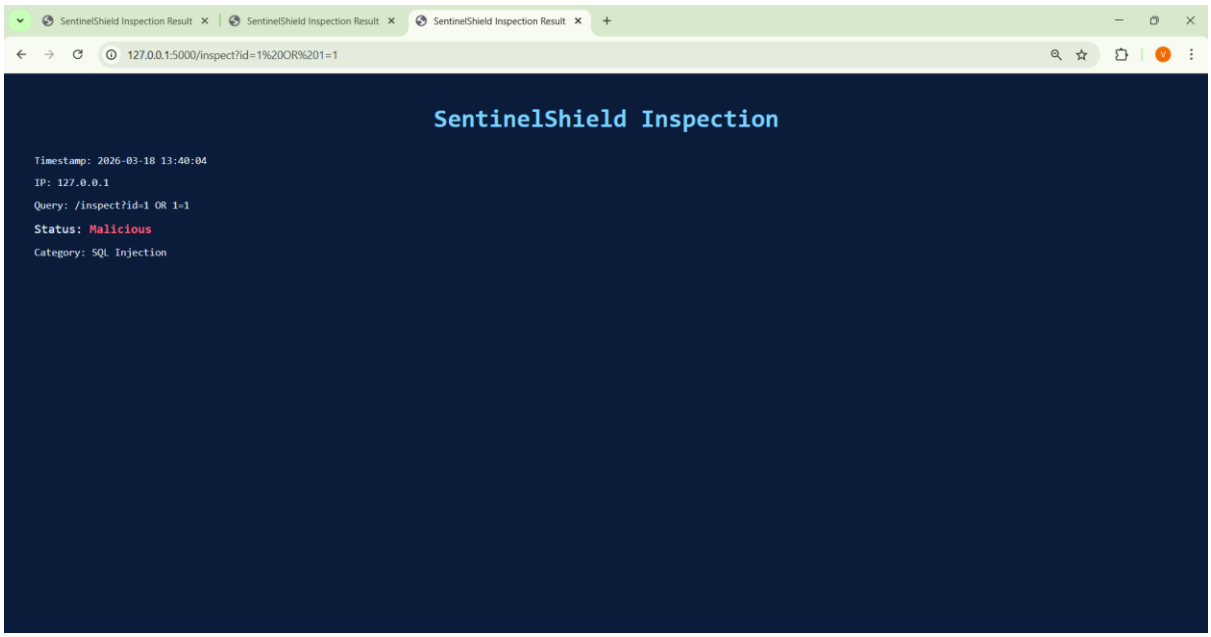


Figure 6: SQL Injection Detection

5. Directory Traversal

http://127.0.0.1:5000/inspect?file=../../etc/passwd

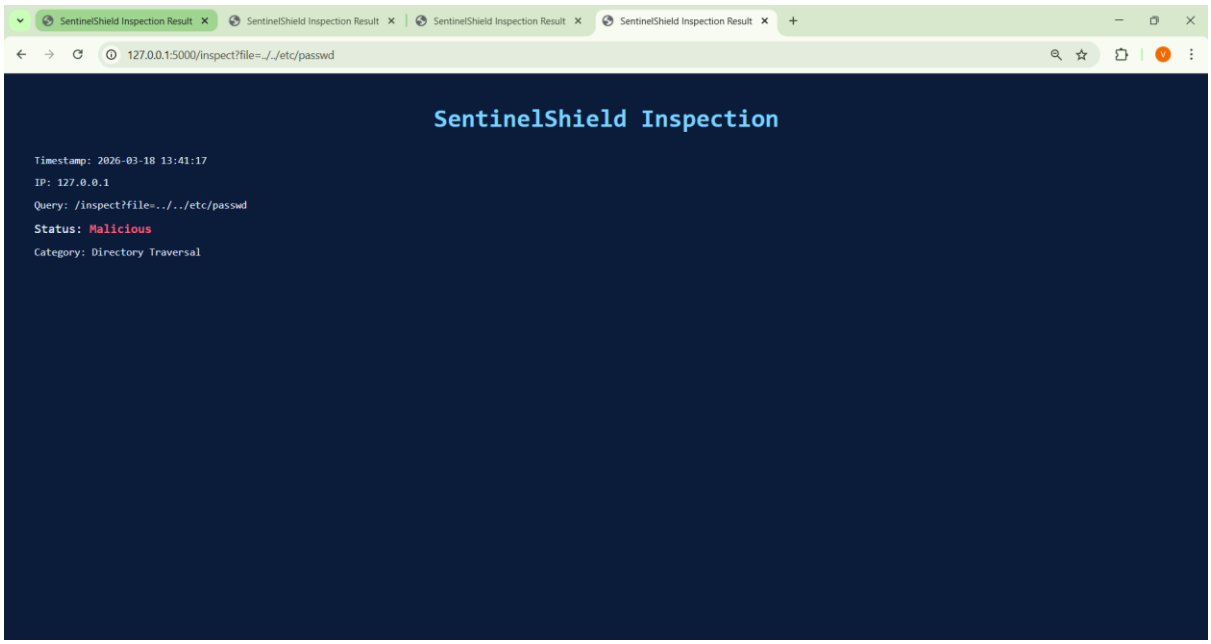


Figure 7: Directory Traversal Detection

6. Command Injection

http://127.0.0.1:5000/inspect?cmd=whoami

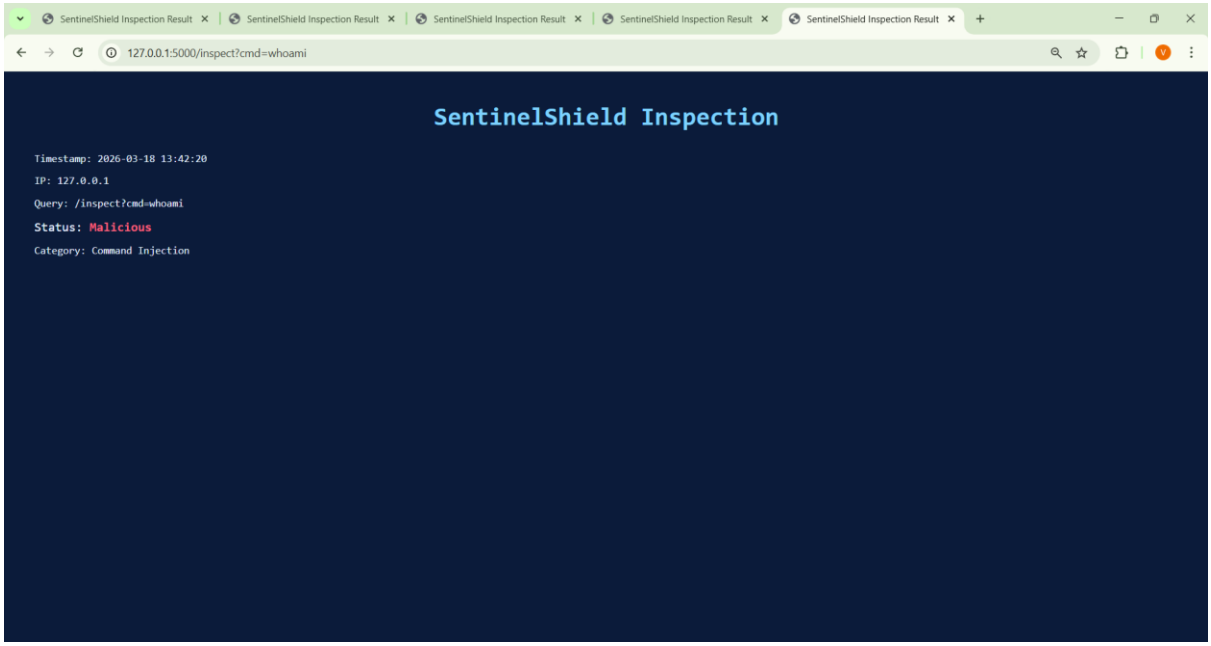


Figure 8: Command Injection Detection

7. Rate Limiting / Brute Force Detection
http://127.0.0.1:5000/inspect?search=test (repeated requests)

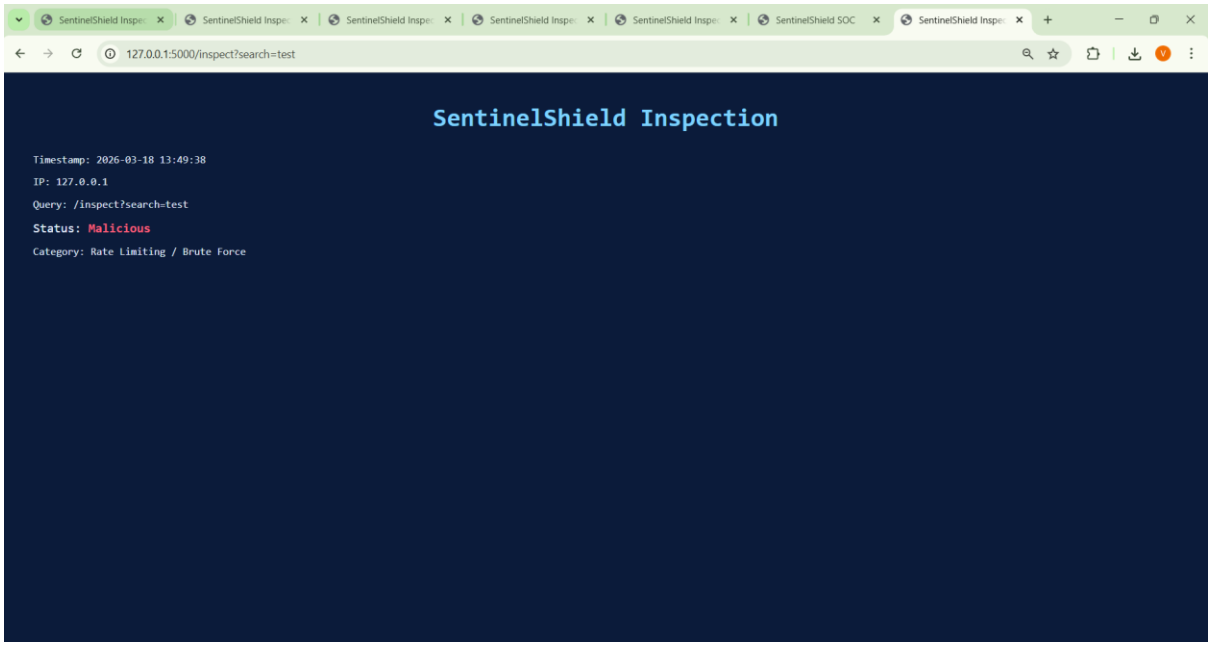


Figure 9: Rate Limiting / Brute Force Detection

13. OBSERVATIONS / ANALYSIS

During testing the following observations were made:

During the testing phase, several observations were made regarding the behavior and performance of the SentinelShield system. Different types of HTTP requests were sent to the application in order to evaluate how accurately the system could detect malicious inputs and monitor request activity.

• **Normal requests passed without triggering alerts**
When standard HTTP requests with normal input parameters were sent to the system, they were successfully processed and classified as normal requests. These requests did not trigger any security alerts, indicating that the system does not generate unnecessary warnings for legitimate traffic.

• **Malicious payloads were correctly detected**
When requests containing malicious patterns such as <script>, or 1=1, or directory traversal sequences were sent to the system, they were correctly detected and classified under the appropriate attack categories. This confirmed that the signature-based detection mechanism was functioning effectively.

• **Security events were successfully recorded in the log file**
All detected events, including both normal and malicious requests, were recorded in the log file. Each log entry contained useful information such as the timestamp, IP address, request query, and attack classification. These logs help in analyzing system activity and identifying potential threats.

• **Rate limiting identified repeated requests from the same IP**
The rate limiting module successfully monitored the number of requests coming from a single IP address within a defined time window. When multiple requests were sent rapidly, the system flagged the activity as suspicious, demonstrating behavior-based detection.

• **The dashboard visualized attack statistics effectively**
The security dashboard displayed summarized statistics such as the number of total requests, malicious requests, and the distribution of different attack types. The charts generated using Chart.js made it easier to understand the system activity and visualize attack trends.

Log analysis and dashboard monitoring together help security analysts understand attack patterns, monitor traffic behavior, and investigate potential security incidents.

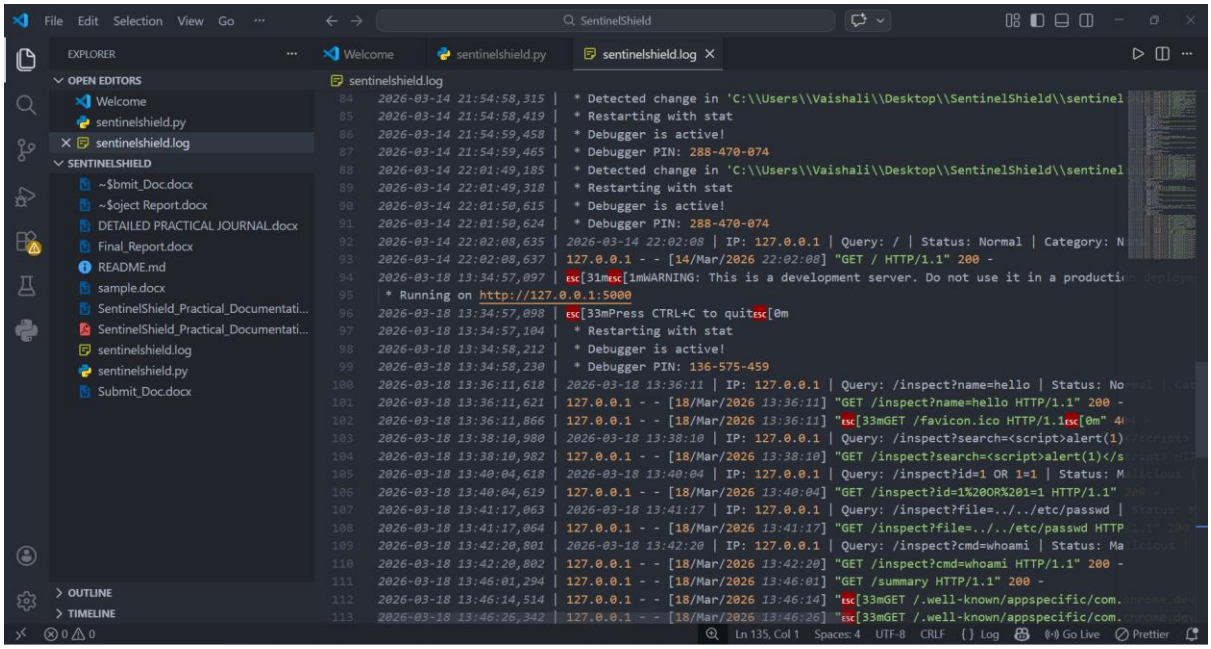
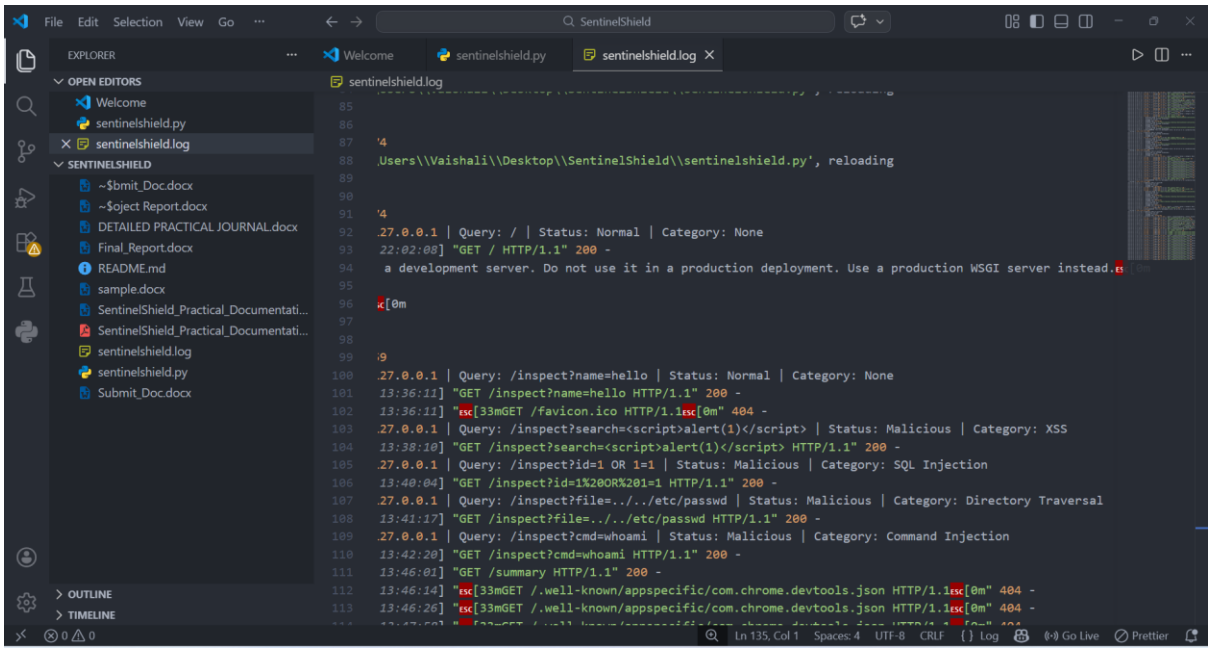
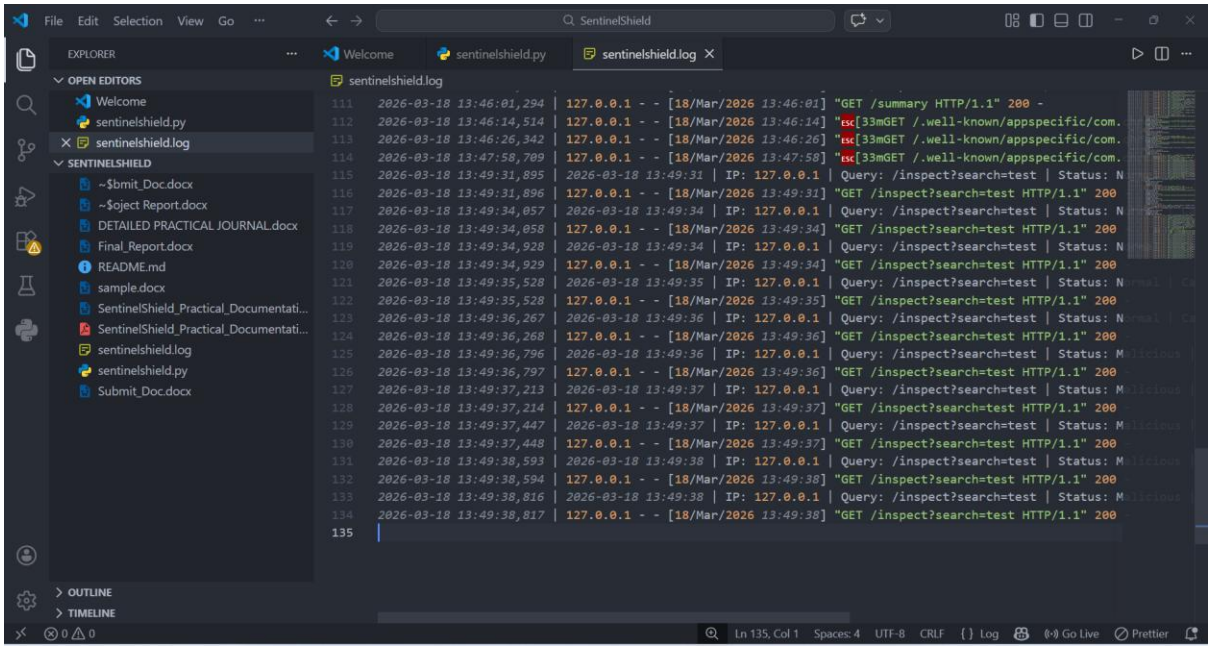
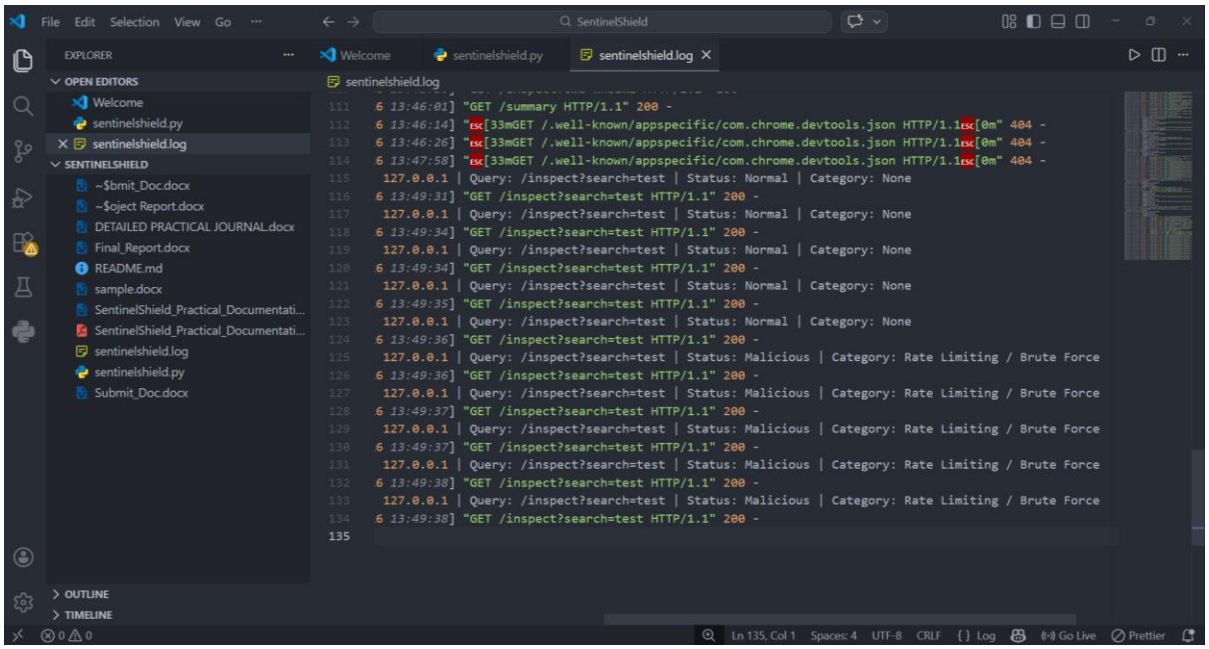


Figure 10: Security Log Entries




File Edit Selection View Go
SentinelShield
EXPLORER
OPEN EDITORS
Welcome
sentinelshield.py
sentinelshield.log
SENTINELSHIELD
~$bmit_Doc.docx
~$oject Report.docx
DETAILED PRACTICAL JOURNAL.docx
Final_Report.docx
README.md
sample.docx
SentinelShield_Practical_Documentati...
SentinelShield_Practical_Documentati...
sentinelshield.log
sentinelshield.py
Submit_Doc.docx
OUTLINE
TIMELINE
sentinelshield.log
2026-03-18 13:46:01,294 | 127.0.0.1 - - [18/Mar/2026 13:46:01] "GET /summary HTTP/1.1" 200 -
2026-03-18 13:46:14,514 | 127.0.0.1 - - [18/Mar/2026 13:46:14] "[33mGET /.well-known/appspecific/com.
2026-03-18 13:46:26,342 | 127.0.0.1 - - [18/Mar/2026 13:46:26] "[33mGET /.well-known/appspecific/com.
2026-03-18 13:47:58,709 | 127.0.0.1 - - [18/Mar/2026 13:47:58] "[33mGET /.well-known/appspecific/com.
2026-03-18 13:49:31,895 | 2026-03-18 13:49:31 | IP: 127.0.0.1 | Query: /inspect?search=test | Status: N
2026-03-18 13:49:31,896 | 127.0.0.1 - - [18/Mar/2026 13:49:31] "GET /inspect?search=test HTTP/1.1" 200
2026-03-18 13:49:34,057 | 2026-03-18 13:49:34 | IP: 127.0.0.1 | Query: /inspect?search=test | Status: N
2026-03-18 13:49:34,058 | 127.0.0.1 - - [18/Mar/2026 13:49:34] "GET /inspect?search=test HTTP/1.1" 200
2026-03-18 13:49:34,928 | 2026-03-18 13:49:34 | IP: 127.0.0.1 | Query: /inspect?search=test | Status: N
2026-03-18 13:49:34,929 | 127.0.0.1 - - [18/Mar/2026 13:49:34] "GET /inspect?search=test HTTP/1.1" 200
2026-03-18 13:49:35,528 | 2026-03-18 13:49:35 | IP: 127.0.0.1 | Query: /inspect?search=test | Status: N
2026-03-18 13:49:35,528 | 127.0.0.1 - - [18/Mar/2026 13:49:35] "GET /inspect?search=test HTTP/1.1" 200
2026-03-18 13:49:36,267 | 2026-03-18 13:49:36 | IP: 127.0.0.1 | Query: /inspect?search=test | Status: N
2026-03-18 13:49:36,268 | 127.0.0.1 - - [18/Mar/2026 13:49:36] "GET /inspect?search=test HTTP/1.1" 200
2026-03-18 13:49:36,796 | 2026-03-18 13:49:36 | IP: 127.0.0.1 | Query: /inspect?search=test | Status: M
2026-03-18 13:49:36,797 | 127.0.0.1 - - [18/Mar/2026 13:49:36] "GET /inspect?search=test HTTP/1.1" 200
2026-03-18 13:49:37,213 | 2026-03-18 13:49:37 | IP: 127.0.0.1 | Query: /inspect?search=test | Status: M
2026-03-18 13:49:37,214 | 127.0.0.1 - - [18/Mar/2026 13:49:37] "GET /inspect?search=test HTTP/1.1" 200
2026-03-18 13:49:37,447 | 2026-03-18 13:49:37 | IP: 127.0.0.1 | Query: /inspect?search=test | Status: M
2026-03-18 13:49:37,448 | 127.0.0.1 - - [18/Mar/2026 13:49:37] "GET /inspect?search=test HTTP/1.1" 200
2026-03-18 13:49:38,593 | 2026-03-18 13:49:38 | IP: 127.0.0.1 | Query: /inspect?search=test | Status: M
2026-03-18 13:49:38,594 | 127.0.0.1 - - [18/Mar/2026 13:49:38] "GET /inspect?search=test HTTP/1.1" 200
2026-03-18 13:49:38,816 | 2026-03-18 13:49:38 | IP: 127.0.0.1 | Query: /inspect?search=test | Status: M
2026-03-18 13:49:38,817 | 127.0.0.1 - - [18/Mar/2026 13:49:38] "GET /inspect?search=test HTTP/1.1" 200
Ln 135, Col 1 Spaces: 4 UTF-8 CRLF { } Log Go Live Prettier


File Edit Selection View Go
SentinelShield
EXPLORER
OPEN EDITORS
Welcome
sentinelshield.py
sentinelshield.log
SENTINELSHIELD
~$bmit_Doc.docx
~$oject Report.docx
DETAILED PRACTICAL JOURNAL.docx
Final_Report.docx
README.md
sample.docx
SentinelShield_Practical_Documentati...
SentinelShield_Practical_Documentati...
sentinelshield.log
sentinelshield.py
Submit_Doc.docx
OUTLINE
TIMELINE
sentinelshield.log
6 13:46:01] "GET /summary HTTP/1.1" 200 -
6 13:46:14] "[33mGET /.well-known/appspecific/com.chrome.devtools.json HTTP/1.1[0m" 404 -
6 13:46:26] "[33mGET /.well-known/appspecific/com.chrome.devtools.json HTTP/1.1[0m" 404 -
6 13:47:58] "[33mGET /.well-known/appspecific/com.chrome.devtools.json HTTP/1.1[0m" 404 -
127.0.0.1 | Query: /inspect?search=test | Status: Normal | Category: None
6 13:49:31] "GET /inspect?search=test HTTP/1.1" 200 -
127.0.0.1 | Query: /inspect?search=test | Status: Normal | Category: None
6 13:49:34] "GET /inspect?search=test HTTP/1.1" 200 -
127.0.0.1 | Query: /inspect?search=test | Status: Normal | Category: None
6 13:49:34] "GET /inspect?search=test HTTP/1.1" 200 -
127.0.0.1 | Query: /inspect?search=test | Status: Normal | Category: None
6 13:49:35] "GET /inspect?search=test HTTP/1.1" 200 -
127.0.0.1 | Query: /inspect?search=test | Status: Normal | Category: None
6 13:49:36] "GET /inspect?search=test HTTP/1.1" 200 -
127.0.0.1 | Query: /inspect?search=test | Status: Malicious | Category: Rate Limiting / Brute Force
6 13:49:36] "GET /inspect?search=test HTTP/1.1" 200 -
127.0.0.1 | Query: /inspect?search=test | Status: Malicious | Category: Rate Limiting / Brute Force
6 13:49:37] "GET /inspect?search=test HTTP/1.1" 200 -
127.0.0.1 | Query: /inspect?search=test | Status: Malicious | Category: Rate Limiting / Brute Force
6 13:49:37] "GET /inspect?search=test HTTP/1.1" 200 -
127.0.0.1 | Query: /inspect?search=test | Status: Malicious | Category: Rate Limiting / Brute Force
6 13:49:38] "GET /inspect?search=test HTTP/1.1" 200 -
127.0.0.1 | Query: /inspect?search=test | Status: Malicious | Category: Rate Limiting / Brute Force
6 13:49:38] "GET /inspect?search=test HTTP/1.1" 200 -
Ln 135, Col 1 Spaces: 4 UTF-8 CRLF { } Log Go Live Prettier

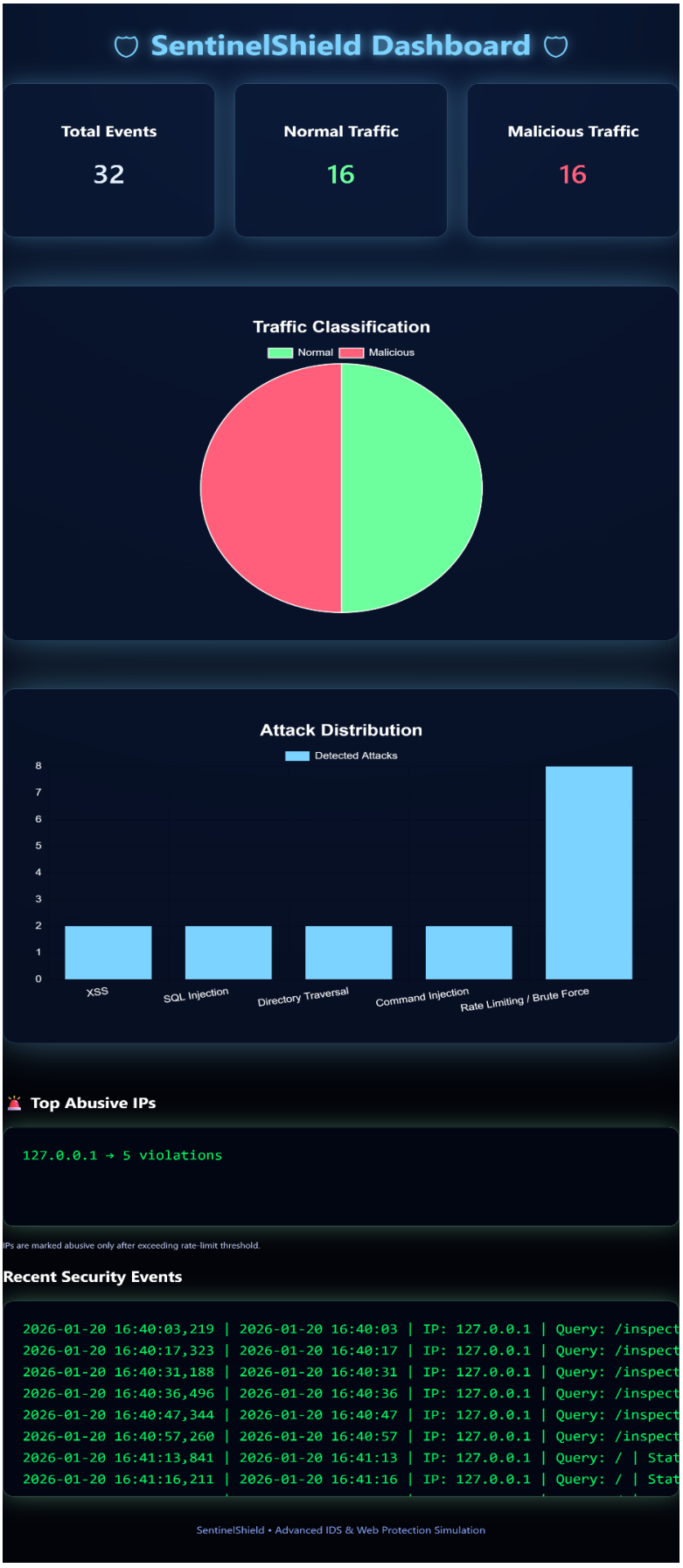


Figure 11: SentinelShield Dashboard

14. ADVANTAGES OF THE SYSTEM

The SentinelShield system provides several advantages in demonstrating the basic concepts of web security monitoring and intrusion detection. These features help in understanding how modern security systems analyze and detect malicious activities in web applications.

• **Detects common web attack patterns**
The system can identify common web attacks such as Cross-Site Scripting (XSS), SQL Injection, Directory Traversal, and Command Injection by analyzing request parameters and matching them with predefined attack signatures.

• **Logs security events for monitoring**
All incoming requests and detection results are recorded in a log file. These logs help in tracking system activity, identifying suspicious behavior, and analyzing potential security threats.

• **Provides visual dashboard statistics**
The system includes a dashboard that displays summarized information about system activity. Graphical charts show statistics such as total requests, malicious traffic, and attack distribution, making it easier to understand the security status of the system.

• **Demonstrates real-world intrusion detection concepts**
SentinelShield helps demonstrate how intrusion detection systems monitor web traffic and detect malicious activities. This provides practical insight into cybersecurity monitoring techniques used in real-world environments.

• **Easy to implement and understand**
The system is built using simple technologies such as Python and Flask, making it easy to implement, test, and understand. This makes the project suitable for educational and learning purposes.

15. LIMITATIONS / DISADVANTAGES

Although the SentinelShield system successfully demonstrates the basic principles of intrusion detection and web security monitoring, it also has certain limitations. These limitations arise mainly because the system is designed as a simplified educational project rather than a full-scale security solution.

• **Relies on predefined attack signatures**
The system detects attacks by comparing incoming requests with predefined malicious patterns. This means that it can only identify attacks that match the existing signatures stored in the detection logic.

• **Cannot detect unknown or zero-day attacks**
Since the system uses signature-based detection, it may not detect new or previously unknown attack techniques that are not included in the predefined patterns.

• **Designed primarily for educational demonstration**
SentinelShield is developed mainly to demonstrate the working principles of intrusion detection systems. Therefore, it does not include advanced security features that are typically found in professional cybersecurity tools.

16. FUTURE SCOPE

The SentinelShield system can be further enhanced by adding additional security features and improving its detection capabilities. Future improvements can help make the system more advanced and closer to real-world intrusion detection solutions.

• **Adding Local File Inclusion detection**
Future versions of the system can include detection mechanisms for Local File Inclusion (LFI) attacks, where attackers attempt to access sensitive files on the server by manipulating file paths.

• **Implementing machine learning-based attack detection**
Machine learning techniques can be integrated to analyze request patterns and identify suspicious behavior automatically. This would allow the system to detect unknown or previously unseen attack patterns.

• **Integrating automated alert systems**
The system can be enhanced to generate automated alerts such as email notifications when malicious activity is detected. This would help administrators respond quickly to potential security threats.

• **Connecting with threat intelligence feeds**
Integration with external threat intelligence sources can help the system stay updated with the latest attack signatures and known malicious IP addresses.

• **Improving dashboard analytics and visualization**
The security dashboard can be improved by adding more detailed charts, graphs, and real-time monitoring features to provide better insights into system activity and detected threats.

17. LEARNING OUTCOMES

This project provided valuable learning experience in the field of cybersecurity and web application security. By developing and testing the SentinelShield system, practical knowledge was gained about how web security monitoring systems analyze incoming requests and detect potential threats.

The key learning outcomes from this project include:

• **Understanding HTTP request analysis**
The project helped in understanding how HTTP requests are structured and how request parameters can be inspected to identify suspicious or malicious inputs.

• **Detecting malicious payloads**
Through testing different attack scenarios, the system demonstrated how malicious payloads such as script injections or SQL commands can be detected within user inputs.

• **Implementing signature-based detection techniques**
The project provided practical experience in implementing signature-based detection methods by comparing incoming request data with predefined malicious patterns.

• **Monitoring logs for attack investigation**
The logging mechanism helped illustrate how system logs can be used to track request activity, identify attack attempts, and analyze potential security incidents.

• **Visualizing security events using dashboards**
The project also demonstrated how dashboards can be used to present security data visually, making it easier to understand system activity and detected threats.

18. CONCLUSION

The SentinelShield project successfully demonstrates the fundamental working principles of an intrusion detection and web protection system. The system analyzes incoming HTTP requests, identifies malicious patterns within request parameters, and monitors user behavior to detect suspicious activities.

Through the implementation of signature-based detection and behavior-based monitoring techniques, the system is able to identify common web attacks such as Cross-Site Scripting, SQL Injection, Directory Traversal, and Command Injection. All detected events are recorded in a log file, which helps in tracking system activity and investigating potential security incidents.

In addition to detection and logging, the SentinelShield system provides a security dashboard that visually presents system statistics, including normal requests, malicious requests, and attack distribution. This visualization helps in understanding overall system activity and identifying attack trends more effectively.

The development and testing of this project provided practical exposure to important cybersecurity concepts such as HTTP request analysis, attack detection mechanisms, log monitoring, and security visualization. These concepts are widely used in real-world cybersecurity environments.

19. PROJECT DELIVERABLES

The following deliverables were produced as part of this project:

- SentinelShield Python application code
- Security log file
- Security monitoring dashboard
- System architecture and workflow diagrams
- Practical project documentation report

20. REFERENCES / BIBLIOGRAPHY

1. OWASP Top 10 Web Application Security Risks – https://owasp.org

2. Flask Official Documentation – https://flask.palletsprojects.com

3. Python Logging Module Documentation – https://docs.python.org

4. Chart.js Official Documentation – https://www.chartjs.org